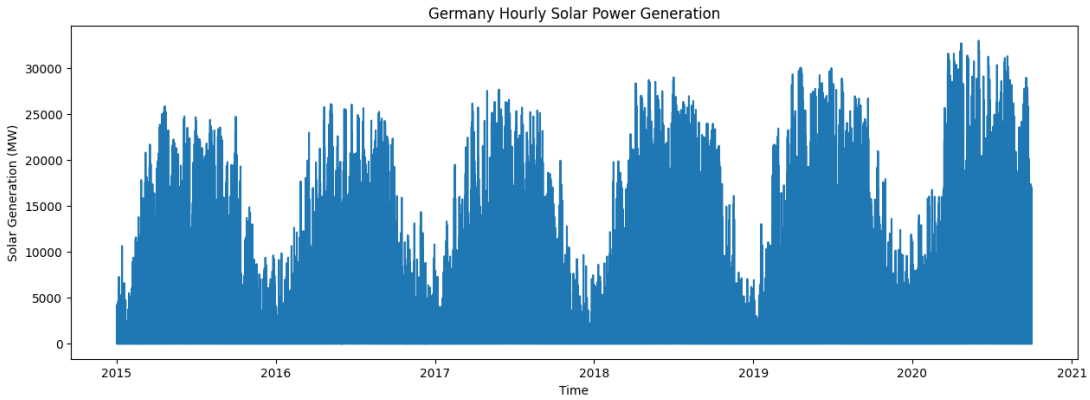
HANDLING MULTIPLE SEASONALITIES IN HOURLY SOLAR POWER GENERATION

Time-series analysis and forecasting using Facebook Prophet



Historical German hourly solar-generation series (MW).

Applied Time Series Analysis — Project Report and Presentation

Ryan (S041) | Yugraj (S043) | Aleric (S050)

MBA Tech Data Science · Semester V · NMIMS MPSTME

Academic year 2026–27

Project synopsis

We examined hourly solar power generation in Germany to understand its daily and yearly patterns. After exploring the data and its autocorrelation, we compared a previous-day forecast with three Prophet models. Including daily and yearly seasonality substantially improved Prophet's accuracy. The previous-day baseline nevertheless had the lowest errors under the evaluation used in this study.

1. Research question and approach

Solar power generation follows a clear daily cycle: output is low at night, increases after sunrise, reaches a daytime peak, and falls again in the evening. Generation also changes throughout the year as daylight hours and weather conditions vary. These daily and annual patterns make solar generation a useful case for studying multiple seasonalities. Our research question was: how do Prophet forecast errors change when daily and yearly seasonal components are added?

We began by importing the German hourly solar-generation data and checking timestamps and missing values. We then studied hourly and monthly profiles and used the autocorrelation function (ACF) to examine recurring patterns. A seasonal-naive forecast used observations from the same hour on the previous day. Finally, we fitted three Prophet configurations using the same pre-2019 training period and evaluated them on 2019 with mean absolute error (MAE) and root mean squared error (RMSE).

Source and files

The data came from Open Power System Data (OPSD), Time Series package dated 6 October 2020. We used the file time_series_60min_singleindex_filtered.csv and selected utc_timestamp and DE_solar_generation_actual. The target represents Germany's aggregated solar electricity generation in megawatts (MW), rather than the output of an individual solar panel or farm.

2. Dataset preparation

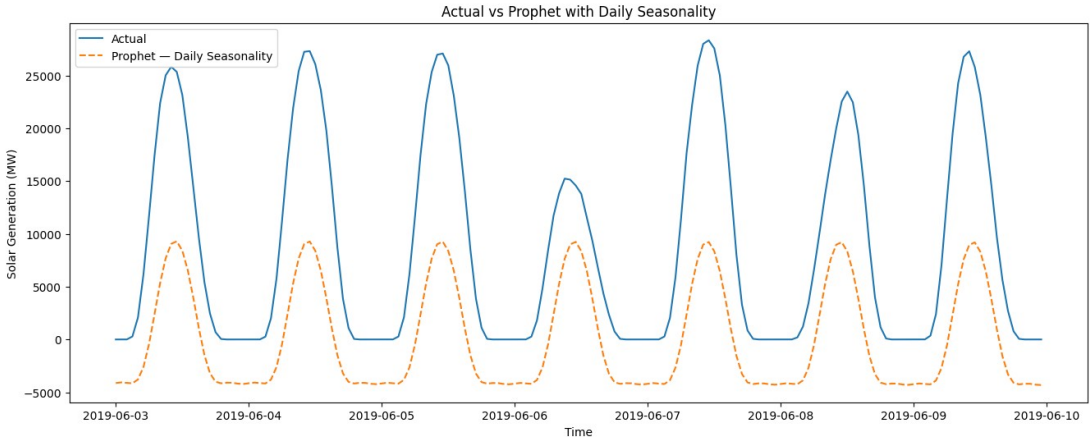
The dataset covers late December 2014 through September 2020 and contains 50,401 hourly timestamps. We retained the timestamp and solar-generation columns, sorted the observations chronologically, and renamed the columns ds (date and time) and y (generation), as required by Prophet.

```
df = pd.read_csv("time_series_60min_singleindex_filtered.csv",
                 usecols=["utc_timestamp", "DE_solar_generation_actual"])
solar = df.rename(columns={"utc_timestamp": "ds", "DE_solar_generation_actual": "y"})
solar["ds"] = pd.to_datetime(solar["ds"], utc=True)
```

Data-quality checks

Check	Observed result
Number of timestamps	50,401
Missing solar values	104 (0.21%)
Duplicate timestamps	0
Unexpected gaps	0; each timestamp is spaced by one hour

Solar generation is often zero at night; these are valid recorded values, not missing data. For Prophet fitting, we excluded rows with missing target observations. For the seasonal-naive forecast, we kept the original hourly sequence intact before shifting the values by 24 hours.



The graph compares actual hourly solar generation during a week in June 2019 with the forecast from the daily-seasonality Prophet model. Although both curves follow a repeating daily pattern, the predicted peaks are much lower than several observed peaks. This illustrates why capturing the daily shape alone is not enough to forecast the full variation in generation.

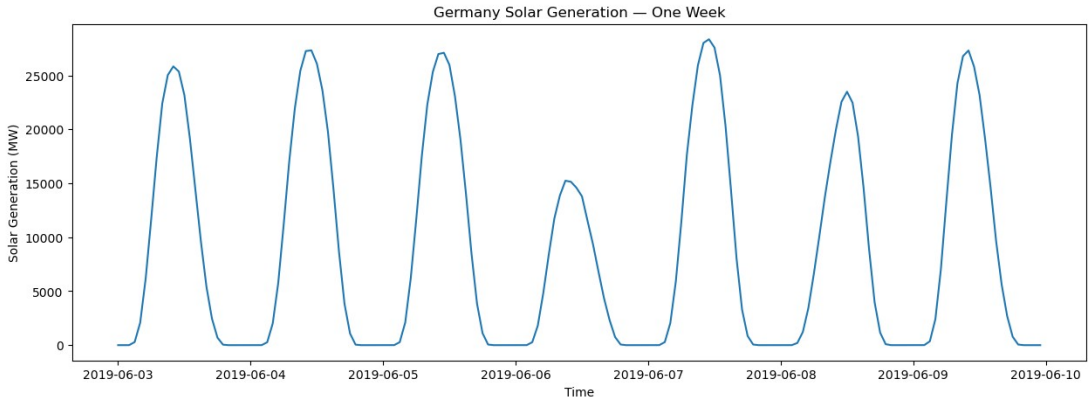


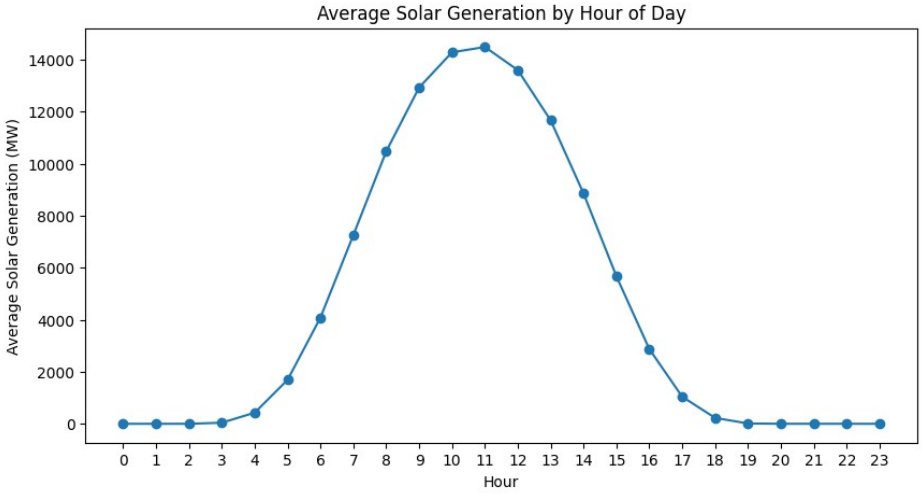
Figure 1. Actual Generation and Daily-Seasonality Prophet Forecast, 3–9 June 2019

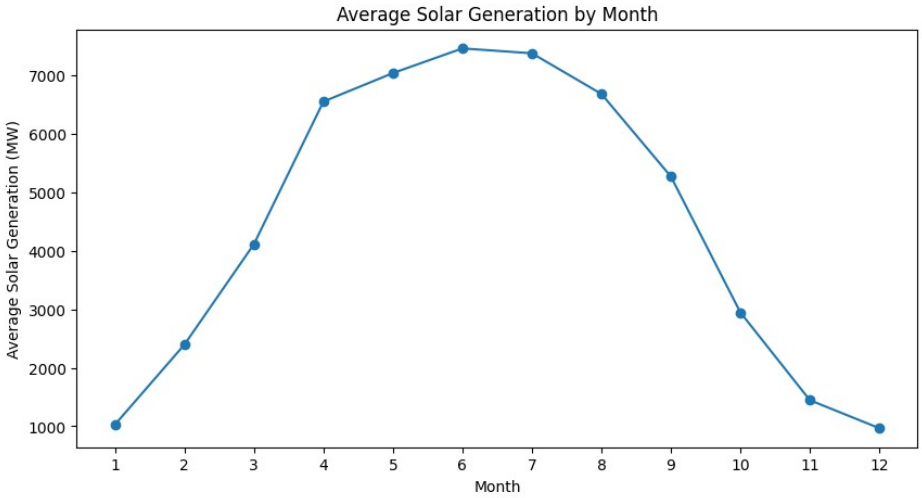
The observed generation also changes from day to day: the timing of the peaks is broadly similar, but their heights differ. A repeating seasonal pattern therefore cannot explain every hourly observation.

3. Discovering seasonality

We estimated the seasonality by calculating the average generation for each hour of the day and plotting it. We performed the same procedure for each calendar month.

```
solar["hour"] = solar["ds"].dt.hour
solar["month"] = solar["ds"].dt.month
hourly_profile = solar.groupby("hour")["y"].mean()
monthly_profile = solar.groupby("month")["y"].mean()
```





The hourly profile shows the power generation in daylight hours and the monthly profile reflects higher average power during the summer months.

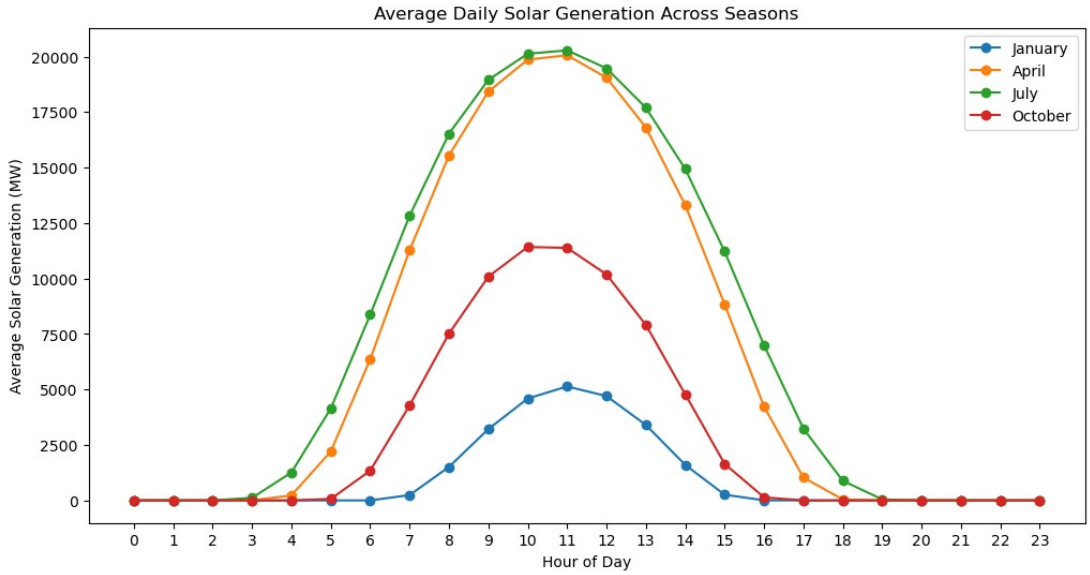


Figure 2. Average Hourly Power-Profiles for January, April, July, and October

The seasonality is visually apparent in these plots, which further illustrate the need for a Prophet model with yearly seasonality. The daily power-profile shapes are similar but the values are lower in the winter and higher in the summer. For the purpose of forecasting, we explored autocorrelation further.

4. Autocorrelation and the daily cycle

We used the autocorrelation function (ACF) to measure the relationship between observations and their earlier values. For hourly data, lag 24 represents the same hour on the previous day. We plotted lags up to 168 hours (one week) and marked the multiples of 24 to make the daily repetition easier to identify.

```
plot_acf(acf_month, lags=168, ax=ax)
for lag in range(24, 169, 24):
    ax.axvline(lag, linestyle="--", alpha=0.3)
```

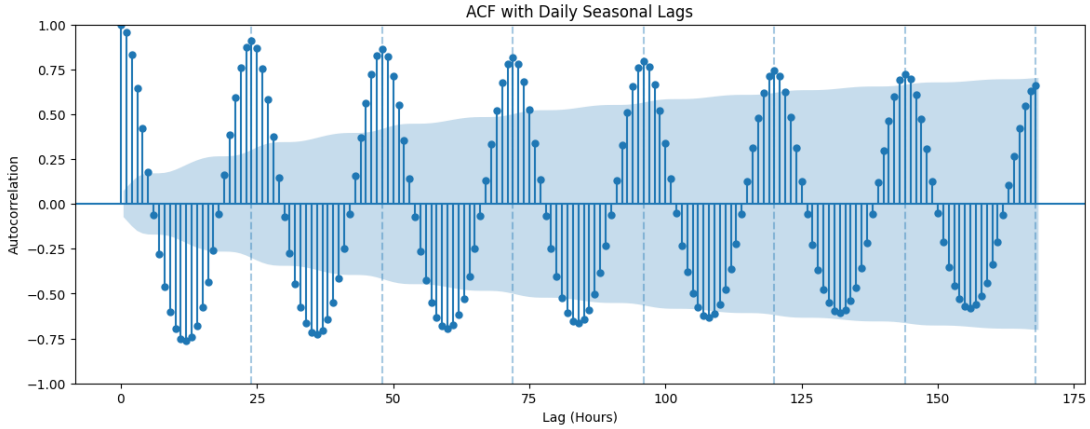


Figure 3. Autocorrelation Plot for Hourly Solar-Generation Data

ACF shows serial dependence and supports the daily seasonal pattern, but it does not establish stationarity. The repeated peaks near lags 24, 48 and 72 agree with the daily cycles visible in the earlier plots. They also support using the same hour on the previous day as a straightforward forecasting baseline.

5. Forecasting setup: baseline and Prophet

Chronological split

We fitted all three Prophet models on observations before 2019 and evaluated them on the full year 2019. We used a chronological split, which is standard for time-series forecasting, so the Prophet models were not trained on future observations from the test period.

```
train = prophet_data[prophet_data["ds"] < "2019-01-01"]
test = prophet_data[(prophet_data["ds"] >= "2019-01-01") &
                    (prophet_data["ds"] < "2020-01-01")]
```

Seasonal-naive baseline

The simplest forecasting method is a seasonal naive model, which moves the values from the previous day by 24 hours. We created this baseline to establish a reasonable comparison against Prophet forecasts.

```
baseline = solar[["ds", "y"]].copy()
baseline["naive_forecast"] = baseline["y"].shift(24)
```

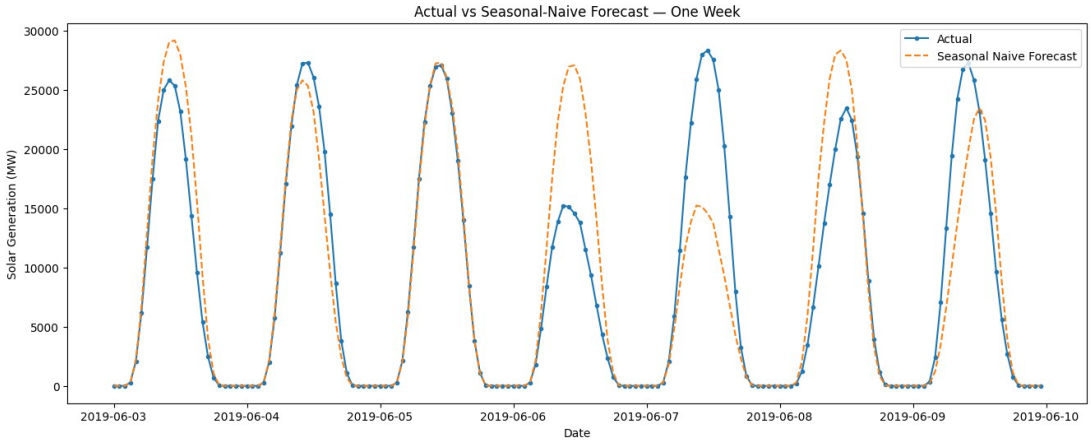


Figure 4. Observed Generation (actual) and Seasonal Naive Forecast for One Week in June 2019

It is critical to remember that the baseline forecast for 2019 used the actual values from the previous day, whereas the Prophet model was fit on the data before 2019 and could not use any observations from the year itself. This difference highlights why a fair comparison between the methods would require a rolling forecast origin.

6. The three Prophet models

Prophet forecasts are based on an underlying trend that is adjusted by seasonal components. We fit three Prophet models, varying the combination of seasonal modes (daily and yearly). All models used the same training set and test data.

Configuration	Daily	Yearly
Trend only	Off	Off
Trend + daily	On	Off
Trend + daily + yearly	On	On

```
model_3 = Prophet(daily_seasonality=True,
                  weekly_seasonality=False,
                  yearly_seasonality=True)
model_3.fit(train)
forecast_3 = model_3.predict(test[["ds"]])
```

We turned off the default weekly seasonality in all models since this project focuses on daily and yearly seasonality. We fit a Prophet model with trend only, then added daily seasonal components, and finally yearly seasonality. This approach allowed us to evaluate how each component affected the forecast accuracy.

7. Results: which method works best?

We evaluated the forecasts using mean absolute error (MAE) and root mean squared error (RMSE), calculated with scikit-learn. Both metrics are expressed in MW, and lower values indicate smaller errors. RMSE gives greater weight to large forecast errors than MAE, so it is useful to report both.

```
mae = mean_absolute_error(test["y"], forecast["yhat"])
rmse = np.sqrt(mean_squared_error(test["y"], forecast["yhat"]))
```

Model	MAE (MW)	RMSE (MW)
Seasonal naive	1,136.62	2,477.54
Prophet: trend only	5,227.64	8,411.00
Prophet: daily	5,794.46	7,077.20
Prophet: daily + yearly	2,907.83	3,762.52

The addition of daily seasonality reduced the Prophet RMSE from 8,411.00 to 7,077.20 MW, but this came at the expense of a higher MAE. The Prophet model with yearly seasonality on top of daily seasonality reduced the RMSE by 47% compared to the daily-only model (from 7,077.20 to 3,762.52). This was the best performing Prophet specification. On the other hand, the seasonal naive model had the lowest MAE and RMSE of all four rows. We decided to report this finding as it is rather than attempting to claim that Prophet is generally better than seasonal naive. The reason is that the seasonal naive forecast used observations from the previous day during 2019, whereas the Prophet model was fit on the data before 2019 and not updated for the year.

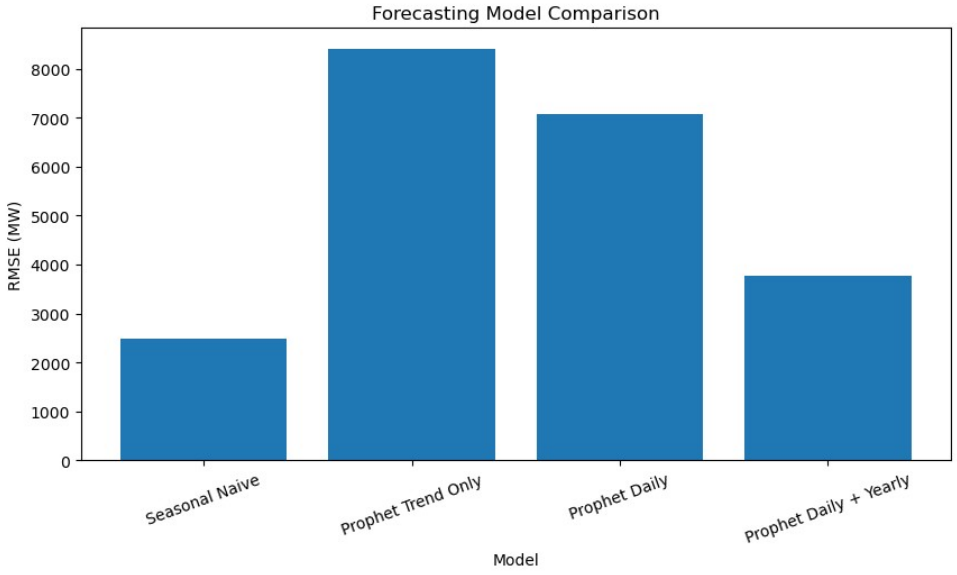


Figure 5. Comparison of RMSE across the Seasonal Naive and Prophet Forecasts

The Prophet model with daily plus yearly seasonality has the lowest MAE and RMSE among the Prophet specifications. However, the seasonal naive rule has the lowest overall errors for the given evaluation task.

8. What the model captured — and missed

The Prophet component plots separate the estimated trend from the annual and within-day seasonal patterns. The yearly component varies markedly across the calendar, while the daily component reflects the typical rise and fall in solar generation through daylight hours. These components describe recurring time-based patterns, not individual weather events.

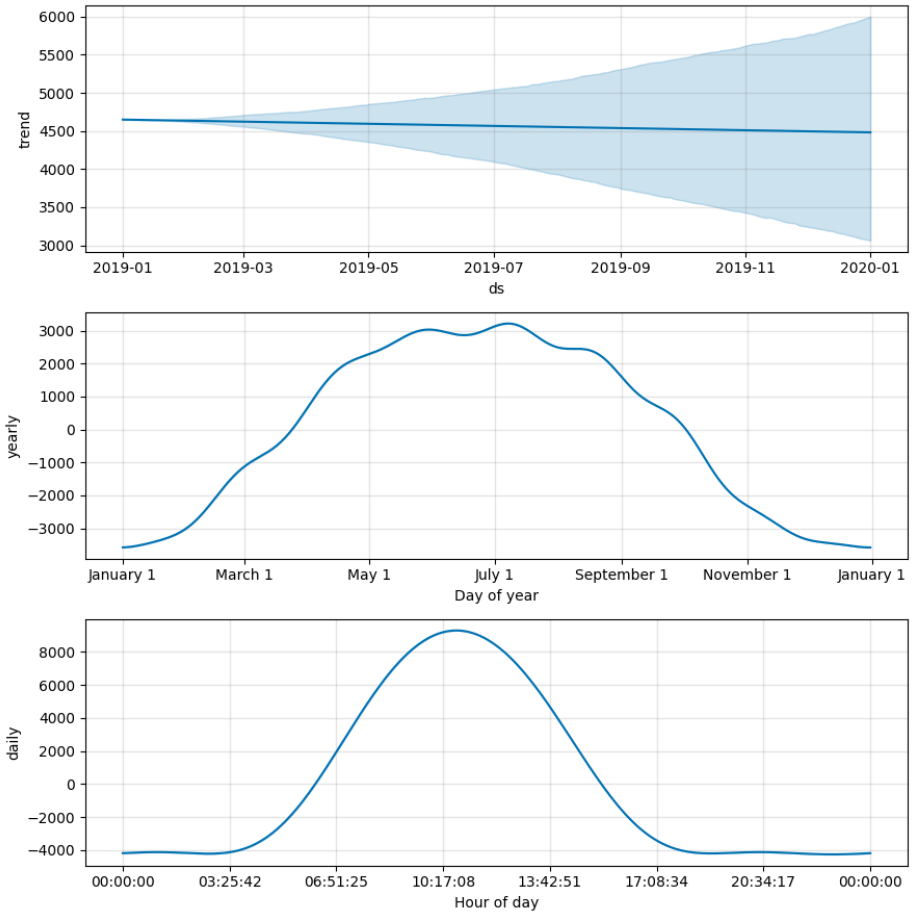


Figure 6. Trend, Annual and Within-day Components Estimated by the Multi-seasonal Prophet Model

Residual checks

```
results_3["residual"] = results_3["y"] - results_3["yhat"]
results_3["hour"] = results_3["ds"].dt.hour
results_3["absolute_error"] = abs(results_3["residual"])
error_by_hour = results_3.groupby("hour")["absolute_error"].mean()
```

The residuals from the Prophet forecast highlight the potential areas for improvement. We first calculated residuals as the differences between the actual and predicted values for the daily plus yearly Prophet model. We visualized the ACF plot for the residuals to check if there was predictable day-to-day variation which the model had failed to capture.

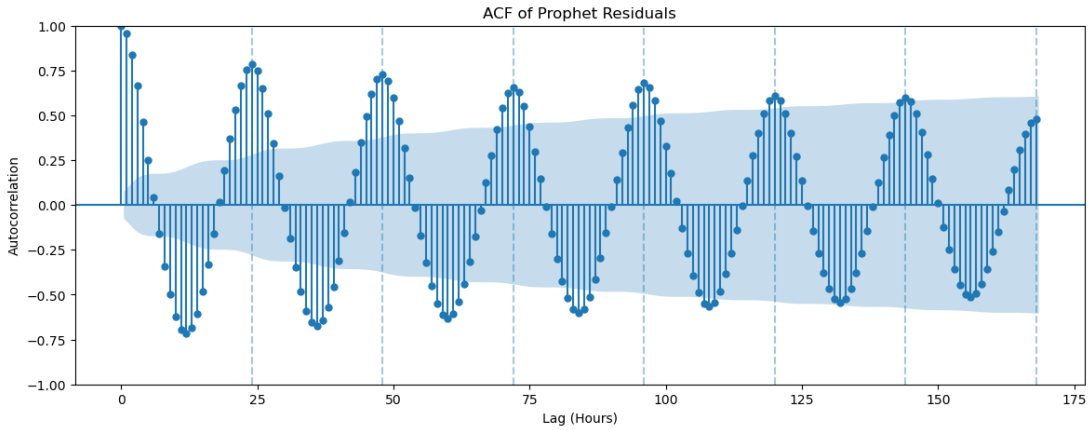


Figure 7. ACF of Residuals for the Daily Plus Yearly Prophet Model.

The ACF plot for the residuals highlights the potential shortcomings in the Prophet model. The dependency between the observations separated by 24, 48, and 72 hours was substantial. This suggests that the Prophet model was not fully capturing the patterns in the data by merely adjusting the trend, daily seasonality and yearly seasonality.

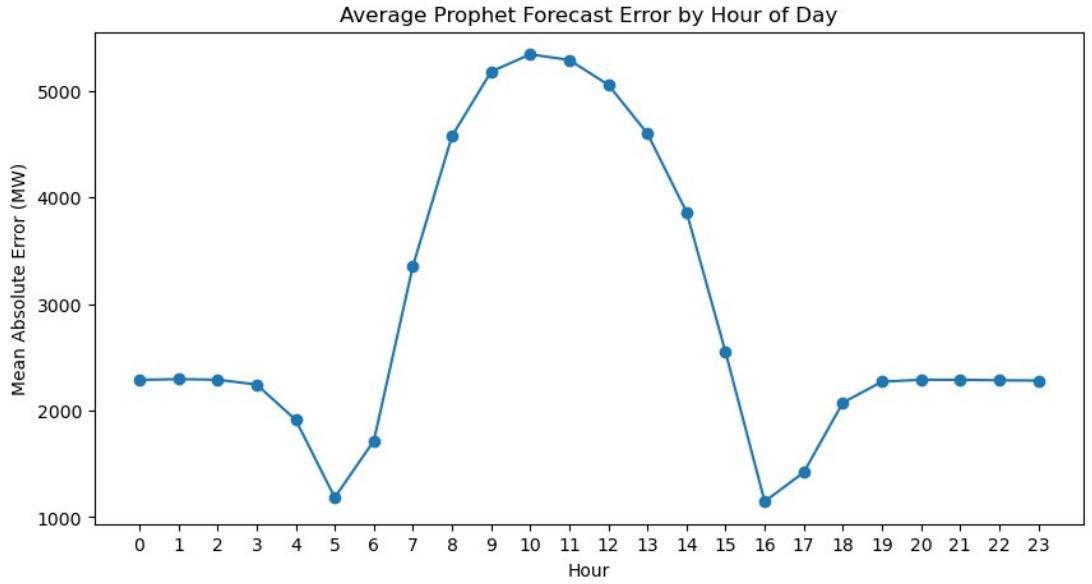


Figure 8. Absolute Prophet Forecast Error for Each Hour of the Day

The mean absolute forecast error is greatest around the daytime generation peak. Errors are smaller during some early-morning and late-afternoon hours, but the plot also shows nonzero errors overnight. The model is based on additive seasonal components and does not automatically enforce zero generation at night.

9. Discussion and conclusion

Summary of insights

The main insight from this experiment was that the hourly electricity generation follows a complex pattern with multiple seasonal cycles. The model with trend only underfit the data and the addition of daily seasonality reduced RMSE, but increased MAE. The Prophet forecast errors were reduced by further incorporating the annual seasonality. The fact that a seasonal naive model still outperformed Prophet has highlighted the value of the previous-day observations for this type of forecasting task.

Limitations of the analysis

The analysis was limited by the choice of predictors. The current Prophet models account for time-related patterns only, whereas additional variables, such as solar irradiance, temperature, and cloud coverage may further improve the forecasts. In addition, the Prophet models did not account for the changes in solar generation due to the growth of the solar power capacity in Germany. Finally, the Prophet and seasonal naive forecasts were not directly comparable because the two methods had different access to the recent observations during evaluation. A proper evaluation of the forecasting methods would require a rolling origin evaluation.

Next steps

A useful extension of this analysis would be to explore the rolling day-ahead forecasts and to incorporate additional variables, such as the weather, into the model. A rolling evaluation would make the comparison between the methods more insightful. An analysis of the day-light errors would also help further understand the patterns in the hourly electricity generation.

Overall, the results show that accounting for daily and annual seasonality substantially improves Prophet forecasts of hourly solar generation. The daily-plus-yearly configuration gave the lowest errors among the Prophet models. However, the previous-day seasonal-naive benchmark performed better under this evaluation, reinforcing the importance of recent observations and a clearly defined forecast horizon. A further study should compare the methods at matching forecast origins and consider weather information alongside time-based patterns.

References

Open Power System Data. Time Series, version 2020-10-06. https://data.open-power-system-data.org/time_series/2020-10-06/

Taylor, S. J., & Letham, B. (2018). Forecasting at scale. The American Statistician, 72 (1), 37–45. Web. https://doi.org/10.1080/00031305.2017.1380080

Prophet documentation. https://facebook.github.io/prophet/docs/quick_start.html